Semana 7: Patrones de Diseño Estructurales — Adapter y Bridge

Curso: Patrones de Diseño de Software (PIS1003)

Metodología: Aprendizaje Basado en Proyectos (ABPr)

Proyecto Integrador: Sistema de Gestión Académica con Arquitectura Basada en Patrones

1. CONTENIDO TEÓRICO

1.1. Ubicación de los Patrones Estructurales

Los patrones estructurales se ocupan de cómo se componen las clases y los objetos para formar estructuras más grandes, manteniendo la flexibilidad y eficiencia. Mientras los patrones creacionales (vistos en el Corte 1) resuelven el cómo crear objetos, los estructurales resuelven el cómo organizar y conectar objetos y clases para que colaboren entre sí.

Los patrones estructurales clásicos del catálogo GoF son:

Patrón	Propósito central
Adapter	Convertir una interfaz en otra que el cliente espera
Bridge	Desacoplar una abstracción de su implementación
Composite	Componer objetos en estructuras de árbol
Decorator	Añadir responsabilidades dinámicamente
Facade	Simplificar el acceso a un subsistema complejo
Flyweight	Compartir estado para soportar muchos objetos
Proxy	Controlar el acceso a un objeto

En esta semana nos centramos en **Adapter y Bridge**, dos patrones que suelen confundirse pero que responden a intenciones muy distintas: **Adapter** resuelve un problema de compatibilidad (algo ya existe y no encaja); **Bridge** resuelve un problema de diseño (algo va a variar en dos dimensiones y queremos evitarlo desde el principio).

1.2. Patrón ADAPTER (Adaptador)

1.2.1. Propósito

Permitir que clases con interfaces incompatibles trabajen juntas**. Adapter** actúa como un intermediario que envuelve un objeto existente (el **Adaptee**) y expone una interfaz que el cliente (**Client**) espera (**Target**), traduciendo las llamadas entre ambas.

Intención GoF: "Convert the interface of a class into another interface clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces."

1.2.2. Motivación: el problema del "enchufe incompatible"

Imagina que tu sistema de gestión académica ya tiene definida una interfaz propia para pasarelas de pago:

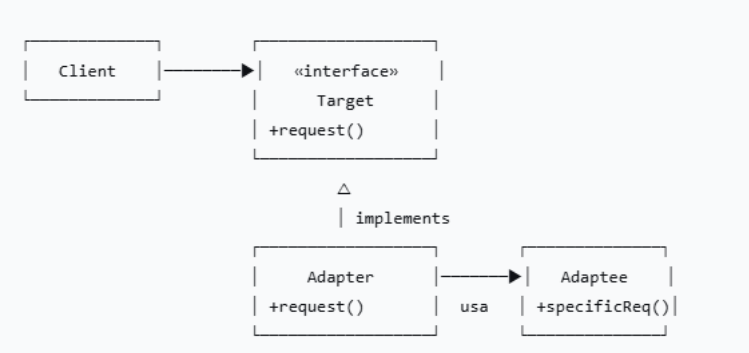
```
public interface PasarelaPago {
    ResultadoPago procesarPago(String idEstudiante, double monto, String concepto);
}
```

Pero la universidad contrata una pasarela externa (por ejemplo, PayU o Stripe) cuya API tiene una firma completamente distinta:

```
public class StripeAPI {
    public StripeCharge createCharge(long amountInCents, String currency, Map<String,Object> metadat
a) { ... }
    public boolean isChargeSuccessful(StripeCharge charge) { ... }
}
```

No puedes (ni quieres) modificar la librería de Stripe, y tampoco quieres contaminar todo tu sistema con su API específica. **Adapter** resuelve exactamente esto.

1.2.3. Estructura (participantes)



Participante	Rol
Target	Interfaz que el cliente conoce y espera
Client	Colabora con objetos que cumplen el Target
Adaptee	Clase existente con interfaz incompatible
Adapter	Envuelve al Adaptee y expone el Target, traduciendo llamadas

1.2.4. Variantes de implementación

a) Adapter de objeto (por composición) — la variante preferida en Java

El **Adapter** contiene una instancia del **Adaptee** y delega en él. Permite adaptar también subclases del **Adaptee**.

```
public class StripeAdapter implements PasarelaPago {
    private final StripeAPI stripe;

    public StripeAdapter(StripeAPI stripe) { this.stripe = stripe; }

    @Override
    public ResultadoPago procesarPago(String idEstudiante, double monto, String concepto) {
        long cents = Math.round(monto * 100);
        Map<String,Object> meta = Map.of("estudiante", idEstudiante, "concepto", concepto);
        StripeCharge charge = stripe.createCharge(cents, "COP", meta);
        return new ResultadoPago(stripe.isChargeSuccessful(charge), charge.getId());
    }
}
```

b) Adapter de clase (por herencia múltiple)

El **Adapter** hereda del **Adaptee** y además implementa el **Target**. En Java solo es posible si el **Target** es una interfaz y el **Adaptee** una clase, ya que Java no tiene herencia múltiple de clases. Es menos flexible porque acopla el **Adapter** a una única jerarquía.

```
public class StripeClassAdapter extends StripeAPI implements PasarelaPago {
    @Override
    public ResultadoPago procesarPago(String idEstudiante, double monto, String concepto) {
        // reutiliza métodos heredados de StripeAPI
        ...
    }
}
```

c) Adapter bidireccional (two-way adapter)

Implementa simultáneamente **Target** y **Adaptee**, permitiendo que objetos de ambos lados se usen indistintamente. Es útil en migraciones graduales.

1.2.5. Consecuencias

Ventajas:

- Reutiliza código legacy o de terceros sin modificarlo.

- Desacopla el cliente de la API concreta del proveedor.

- Permite integrar múltiples proveedores bajo una misma interfaz (PayU, Stripe, MercadoPago → todos como *PasarelaPago*).

Desventajas / trade-offs:

- Añade una capa de indirección (pequeño costo de rendimiento y complejidad).

- Si la interfaz del **Adaptee** cambia, hay que mantener el **Adapter**.

- No debe confundirse con **Facade**: **Adapter** convierte interfaces; **Facade** simplifica un subsistema.

1.2.6. Cuándo usar Adapter

- Integración de librerías legacy o APIs de terceros con interfaces incompatibles.

- Migraciones graduales: el sistema nuevo convive con el viejo.

- Cuando quieres que varias implementaciones (PayU, Stripe, PayPal) sean intercambiables bajo una interfaz propia.

- No lo uses si controlas ambos lados del código: es mejor refactorizar para que coincidan directamente.

1.2.7. Ejemplos en el mundo real

- **JDBC**: *java.sql.Connection* es el Target; cada driver (MySQL, PostgreSQL) trae su propio **Adapter**.

- *InputStreamReader*: adapta un *InputStream* (bytes) a un *Reader* (caracteres).

- Spring *HandlerAdapter*: adapta distintos tipos de controladores al *DispatcherServlet*.

- *Arrays.asList()*: adapta un array a la interfaz *List*.

1.3. Patrón BRIDGE (Puente)

1.3.1. Propósito

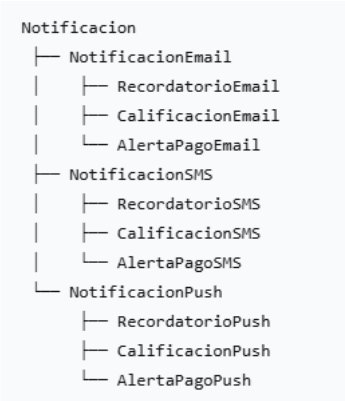
Desacoplar una abstracción de su implementación, de modo que ambas puedan variar independientemente. **Bridge** separa una jerarquía de clases en dos jerarquías paralelas —abstracción e implementación— conectadas por composición en lugar de herencia.

Intención GoF: "Decouple an abstraction from its implementation so that the two can vary independently."

1.3.2. Motivación: la "explosión combinatoria" de subclases

Supón que en el Sistema de Gestión Académica necesitas enviar notificaciones. Tienes varios tipos de notificación (recordatorio de matrícula, calificación publicada, alerta de pago) y varios canales de envío (email, SMS, push móvil).

Si usas herencia pura:



Con 3 tipos × 3 canales = 9 clases. Si mañana se agrega un canal (WhatsApp) → 12. Si se agrega un tipo → 16. Esto es explosión combinatoria y viola Open/Closed Principle.

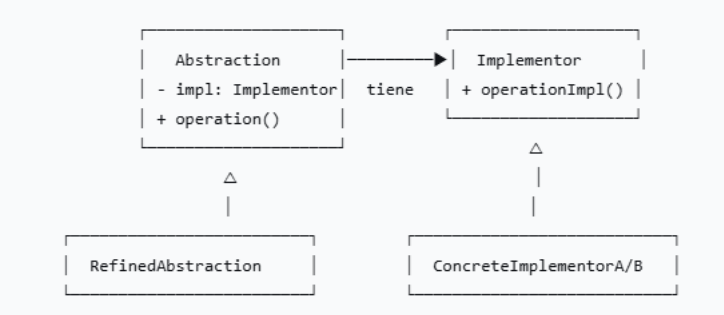
Bridge resuelve esto separando las dos dimensiones en dos jerarquías independientes y combinándolas por composición:

- **Abstracción:** jerarquía de tipos de notificación.

- **Implementador:** jerarquía de canales de envío.

Resultado: 3 + 3 = 6 clases, y agregar un canal o un tipo solo requiere una clase nueva en su jerarquía.

1.3.3. Estructura (participantes)



Participante	Rol
Abstraction	Define la interfaz de alto nivel; mantiene una referencia al Implementor
RefinedAbstraction	Extiende la abstracción con variaciones
Implementor	Define la interfaz de bajo nivel (implementación)
ConcreteImplementor	Implementación concreta de la plataforma/canal

Diferencia clave con **Adapter**: en **Adapter**, el **Adaptee** ya existía con otra interfaz; en **Bridge**, el Implementor se diseña desde el inicio para ser combinado con la abstracción.

1.3.4. Implementación en el proyecto integrador

Dimensión 1 — Implementador (canales de envío):

```
public interface CanalNotificacion {
    void enviar(String destinatario, String asunto, String mensaje);
}
```

```
public class CanalEmail implements CanalNotificacion {
    @Override
    public void enviar(String destinatario, String asunto, String mensaje) {
        // integración con servidor SMTP / SendGrid
    }
}

public class CanalSMS implements CanalNotificacion {
    @Override
    public void enviar(String destinatario, String asunto, String mensaje) {
        // integración con Twilio / proveedor SMS
    }
}

public class CanalPush implements CanalNotificacion {
    @Override
    public void enviar(String destinatario, String asunto, String mensaje) {
        // integración con Firebase Cloud Messaging
    }
}
```

Dimensión 2 — Abstracción (tipos de notificación):

```
public abstract class Notificacion {
    protected final CanalNotificacion canal;   // el "puente"

    protected Notificacion(CanalNotificacion canal) {
        this.canal = canal;
    }

    public abstract void notificar(Estudiante estudiante);
}
```

```
public class RecordatorioMatricula extends Notificacion {
    public RecordatorioMatricula(CanalNotificacion canal) { super(canal); }

    @Override
    public void notificar(Estudiante e) {
        canal.enviar(e.getEmail(), "Recordatorio de matrícula",
                     "Su matrícula vence el " + e.getFechaLimiteMatricula());
    }
}

public class CalificacionPublicada extends Notificacion {
    public CalificacionPublicada(CanalNotificacion canal) { super(canal); }

    @Override
    public void notificar(Estudiante e) {
        canal.enviar(e.getEmail(), "Calificación publicada",
                     "Su nota final es: " + e.getUltimaNota());
    }
}
```

Uso (el cliente compone ambas dimensiones):

```
Estudiante e = repositorio.buscar("2024-001");

Notificacion n1 = new RecordatorioMatricula(new CanalEmail());
Notificacion n2 = new RecordatorioMatricula(new CanalSMS());
Notificacion n3 = new CalificacionPublicada(new CanalPush());

n1.notificar(e);
n2.notificar(e);
n3.notificar(e);
```

Observa cómo cualquier tipo de notificación se combina con cualquier canal sin necesidad de crear clases combinadas.

1.3.5. Consecuencias

Ventajas:

- Elimina la explosión combinatoria de subclases.

- Abstracción e implementación evolucionan independientemente.

- Cumple Open/Closed: agregar canal o tipo no modifica código existente.

- Favorece la composición sobre la herencia (principio GoF).

Desventajas / trade-offs:

- Incrementa la complejidad inicial del diseño (dos jerarquías en lugar de una).

- Si solo hay una dimensión que varía, **Bridge** es over-engineering; conviene usarlo cuando existen dos o más ejes de variación reales.

- Requiere identificar correctamente cuál es la abstracción y cuál el implementador; una mala división produce diseños confusos.

1.3.6. Cuándo usar Bridge

- Cuando una clase tiene dos o más dimensiones ortogonales de variación (tipo × canal, forma × renderizador, figura × API gráfica).

- Cuando quieres evitar un vínculo permanente entre abstracción e implementación (permitir cambiar implementación en runtime).

- Cuando ambas jerarquías deben extenserse por separado sin recompilar la otra.

- Cuando hay herencia profunda que se multiplica combinatoriamente.

- No lo uses si solo una dimensión varía: ahí bastan **herencia** o **Strategy**.

1.3.7. Ejemplos en el mundo real

- **JDBC:** *DriverManager* (abstracción) ↔ drivers concretos (implementadores).

- **AWT/Swing**: componentes (*Component*) ↔ peer nativo por plataforma (*Peer*).

- **APIs de logging:** SLF4J como abstracción ↔ Logback/Log4j como implementación.

- Gráficos: *Shape* (abstracción) ↔ *DrawingAPI* (implementador: OpenGL, DirectX, Vulkan).

1.4. Comparativa ADAPTER vs. BRIDGE

Intención	Hacer compatibles interfaces incompatibles	Desacoplar abstracción de implementación
Cuándo se aplica	**Después**: hay código existente que no encaja	**Antes/durante el diseño**: prever dos ejes de variación
Motivación típica	Integrar librería de terceros o legacy	Evitar explosión combinatoria de subclases
Relación entre participantes	Adapter *envuelve* al Adaptee	Abstracción *tiene un* Implementor (puente)
Interfaz del envuelto	Incompatible con el Target	Definida por el diseñador; compatible por diseño
Número de jerarquías	Una (Adapter implementa Target)	Dos (Abstracción e Implementor)
Cambio en runtime	No es su foco	Sí: se puede intercambiar el Implementor
Riesgo principal	Confundirlo con Facade o Decorator	Over-engineering si solo una dimensión varía
Ejemplo del proyecto	`StripeAdapter implements PasarelaPago`	`Notificacion` ↔ `CanalNotificacion`

Regla mnemotécnica:

Adapter = traductor (arregla lo que ya existe).

Bridge = arquitecto (diseña dos caminos paralelos desde el inicio).

1.5. Relación con los principios SOLID

Principio	Cómo lo aplican Adapter y Bridge
Open/Closed	Ambos permiten extender comportamiento sin modificar código cliente
Liskov Substitution	Los adaptadores y abstracciones respetan contratos de sus interfaces
Interface Segregation	Adapter expone solo lo que el cliente necesita; Bridge separa interfaces gruesas en dos
Dependency Inversion	El cliente depende de `PasarelaPago` / `Notificacion` (abstracciones), no de Stripe o SMTP (concreciones)
Single Responsibility	Cada Adapter traduce una sola API; cada ConcreteImplementor cubre un solo canal

1.6. Anti-patrones y errores comunes

1. Adapter innecesario: adaptar algo que podrías refactorizar directamente.

2. Adapter que hace demasiado: si traduce, valida, persiste y notifica, ya no es **Adapter**; es un God Object.

3. Bridge con una sola dimensión: produce dos jerarquías para nada.

4. Confundir Bridge con Strategy: Strategy varía algoritmos en runtime; Bridge varía estructura (abstracción + implementación) en tiempo de diseño.

5. Confundir Adapter con Decorator: Decorator añade responsabilidades manteniendo la misma interfaz; **Adapter** cambia la interfaz.

1.7. Cierre teórico y preguntas de reflexión

- ¿Por qué en el proyecto integrador conviene un *StripeAdapter* en lugar de llamar directamente a *StripeAPI*?

- ¿Qué pasaría si en el sistema de notificaciones usáramos herencia en lugar de **Bridge**? ¿Cuántas clases tendríamos al agregar WhatsApp y un nuevo tipo "Encuesta de satisfacción"?

- ¿Podría **Adapter** y **Bridge** combinarse en el mismo módulo? (Sí: el **CanalEmail** puede internamente usar un **Adapter** para SendGrid).

CIERRE DE LA SESIÓN

Ideas clave para llevarse:

1. **Adapter** = traductor que arregla lo que ya existe. Úsalo al integrar librerías de terceros o legacy.

2. **Bridge** = arquitecto que separa dos dimensiones de variación desde el diseño. Úsalo para evitar explosión combinatoria.

3. Ambos favorecen composición sobre herencia y cumplen Open/Closed.

4. El costo es una capa extra de indirección; se justifica cuando hay variabilidad real.

5. En el proyecto integrador: *Adapter* para pasarelas de pago; *Bridge* para notificaciones y (en el HTI) para reportes.

Próxima semana: Patrón Composite — composición recursiva de objetos en estructuras de árbol (aplicación: estructura académica Facultad → Programa → Curso → Estudiante).